Pair Programming AI Assistant — Audit: Scope & Closures

KK

AUDIT PROMPT

Analyze the `addTask` method in `TaskQueue` for scope and closure behavior:

- What is the lexical scope of the inner `notify()` function, and what variables does it close over?
- What happens when `notify` accesses `name`? Why is this problematic?
- Why should variables be block-scoped with `let` / `const` for predictable code?
- How is the closure formed, and why is defining it inside `addTask` an antipattern?

AI

Scope & Closure Audit Report

1. Lexical Scope and Captured Variables (Closures)

The function `notify()` is declared directly inside the execution context of `addTask(taskFn, priority)`. Because JavaScript uses **lexical scoping**, `notify` retains access to all variables in its parent function's scope even after outer statements execute.

- **Closed-over variables:** `notify` forms a closure over `priority` and `taskFn` from the parameter list of `addTask`.
- It also retains reference to the enclosing scope chain up to the module/global object.

2. The Severe Scope Trap: Unbound `name`

> **Bug Identified:** `console.warn(`High priority task added to ${name}.`);`
>
> In `TaskQueue`, `name` was only a parameter in the `constructor(name)`. It is **NOT** a parameter of `addTask`, nor is it a local variable in scope.
>
> - **Node.js:** Throws an uncaught `ReferenceError: name is not defined` as soon as priority > 9 is enqueued, crashing runtime execution.
> - **Browser:** Falls back to `window.name` (an empty string or browser window title), producing silent bugs rather than using `this.queueName`.